

Vehicle Service Center Management System

OODS Semester Project Proposal

Project Title

Vehicle Service Center Management System

Problem Statement

Manual management of vehicle service centers is inefficient, error-prone, and time-consuming. Service centers face challenges in:

- Tracking customer and vehicle information accurately
- Managing service appointments and scheduling mechanics
- Maintaining service history records
- Generating bills and managing payments
- Retrieving historical data quickly for decision-making
- Sorting and analyzing records based on various parameters (cost, date, priority)

These inefficiencies lead to customer dissatisfaction, delayed services, revenue loss, and poor resource utilization.

Proposed Solution

Develop a **console-based Vehicle Service Center Management System** using **C++** that automates core operations including:

- Customer and vehicle registration
- Appointment scheduling and management
- Mechanic assignment and workload tracking
- Service request processing and status updates
- Billing and payment management
- Historical data retrieval and reporting
- Efficient sorting of records by cost, date, priority, or other criteria

All data will be stored in **text files** for persistence, enabling efficient file handling operations. An optional **REST API** and **frontend interface** may be integrated later for enhanced user experience.

Implementation of OOP and DSA Concepts

Object-Oriented Programming Concepts

1. Classes and Objects

- **Customer:** Stores customer details (ID, name, contact, address)
- **Vehicle:** Stores vehicle information (registration number, model, owner)
- **ServiceRequest:** Represents service bookings (request ID, date, type, status)
- **Mechanic:** Contains mechanic details (ID, name, specialization, availability)
- **Bill:** Manages billing information (service charges, taxes, total amount)

2. Encapsulation

- Private data members with public getter/setter methods
- Validation logic within classes to ensure data integrity

3. Inheritance

- Base class: **Person** for common attributes (name, contact)
- Derived classes: **Customer** and **Mechanic** inherit from **Person**
- Specialized vehicle types like **Car**, **Motorcycle**, **Truck**

4. Polymorphism

- Virtual functions for service types (maintenance, repair, inspection)
- Operator overloading for comparison and display

5. Abstraction

- Abstract base classes for service interfaces
- Hiding complex file operations behind clean methods

Data Structures Used

Data Structure	Use Case
Linked List	Manage dynamic service request queue, customer & vehicle records
Vector/Array	Store mechanic schedules and customer lists
Hash Map	Fast lookup for customer and vehicle data
Queue/Priority Queue	Appointment scheduling and urgent requests
Stack	Undo operations and navigation history
Sorting (Merge Sort)	Sorting records by cost, date, priority, workload, or spending

Member	Role	Responsibilities
Shaheer Afzal Butt (24i-6131)	Project Lead & Core Developer	Architecture design, file I/O, Customer/Vehicle modules, Merge Sort
Musaab Gul Faraz (24i-6001)	Service Management Developer	Mechanic assignment, scheduling, queues, ServiceRequest module
Haidar Rahim (24i-6189)	Billing & Reporting Developer	Billing module, reports, analytics, sorting by cost, documentation

Week	Tasks	Deliverables
1	Requirements gathering, UML diagrams	Design document
2	Setup project structure, define classes	Project skeleton
3–4	Implement Customer & Vehicle modules	Functional modules

Week	Tasks	Deliverables
5–6	Implement ServiceRequest & Mechanic modules	Scheduling system
7	Implement Billing & Reporting	Billing + reports
8	Integration & interface development	Complete system
9	Testing & debugging	Stable release
10	Documentation & presentation	Final submission
Optional	REST API + Frontend UI	Web version

Key Features Summary

- Customer & Vehicle Management
- Appointment Scheduling
- Mechanic Tracking
- Service Monitoring
- Automated Billing
- Search & Filter Tools
- Data Persistence
- Merge Sort-Based Record Sorting
- Reporting & Analytics
- (Optional) Web-Based Interface

Conclusion

The **Vehicle Service Center Management System** applies **Object-Oriented Design** and **efficient Data Structures** to automate essential operations of a vehicle service center.

The **console-based system** will include advanced sorting (Merge Sort), reliable file handling, and modular OOP architecture — ensuring efficiency, scalability, and accuracy.

Future improvements can include REST API and a responsive web interface.

Project Type: Console Application (C++)

Duration: 10 Weeks

Team Size: 3 Members

Feasibility: High

Key Algorithm: Merge Sort ($O(n \log n)$)